

Heterogeneous Acceleration by A.I.

Team 1

2026/5/22

Motivation

- The end of Moore's Law has driven the proliferation of heterogeneous systems. e.g. Hardware acceleration via GPUs, CPUs or FPGAs at the same time.
- In assignments, we were only utilizing one type of hardware acceleration.
- We are interested in heterogeneous acceleration with the help of A.I. and compare the performance where we only utilize one type of hardware.

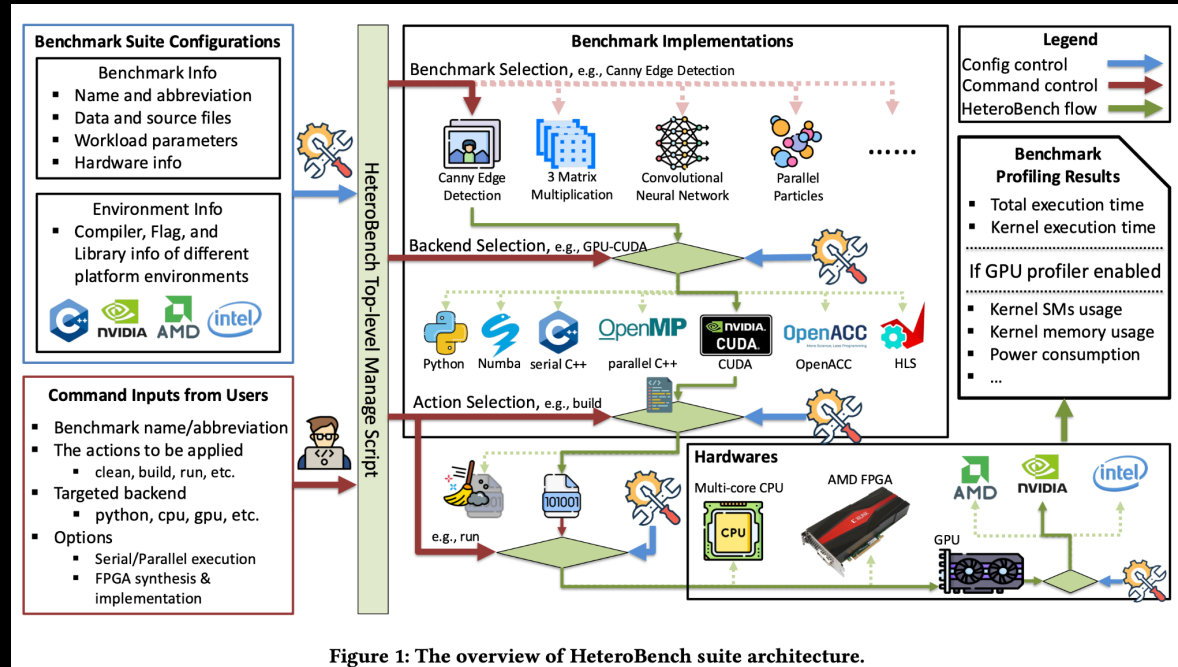
Dataset

- HeteroBench: a benchmark for heterogeneous systems, including the dataset

Benchmarks (abbreviation)	# of Compute Kernels	Application Domain
Canny Edge Detection (ced)	5	Image Processing
Sobel Filter (sbf)	3	
Optical Flow (opf)	8	
Convolutional neural Network (cnn)	5	Artificial Intelligence
Multi-layer Perceptron (mlp)	3	
Digit Recognition (dgr)	2	
One Head Attention (oha)	3	
Spam Filter (spf)	4	
3 Matrix Multiplications (3mm)	3	Numerical Computing
Alternating Direction Implicit (adi)	2	
Parallelize Particle (ppc)	2	Physical Simulation

Dataset

- HeteroBench provides a unified way to run program on cpu or gpu.



Dataset

- The paper shows that if kernels are assigned to accelerators without careful consideration, the performance can be poor.
- We will use all 11 benchmarks to test if AI can make Hetero run better than C++ OpenMP for every benchmark.

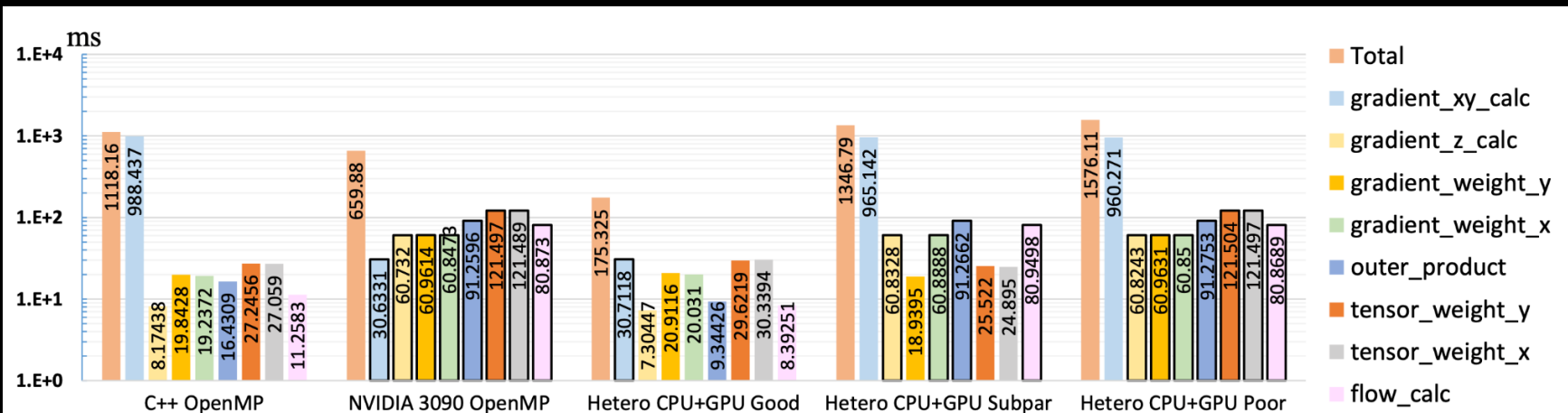


Figure 5: The execution time comparison between homogeneous and heterogeneous implementations of opf in the *Log10* scale for each kernel in a single iteration on NVIDIA RTX 3090. The bars with solid borders denote the kernels run on GPU.

Prompt Engineering

01 Context Setup

- Read HeteroBench README + paper
- Feed hardware spec (CPU & GPU)
- AI generates architecture diagram + design tips

02 Workflow Execution

- Block-ify → Profile → Decide → Execute
- Each step follows a fixed output format (structured SOP)

03 Analysis & Decision

- AI analyzes each kernel's compute intensity
- Produce Roofline model diagrams
- Decide CPU vs GPU assignment per kernel

04 Unit Test & Feedback

- Validate correctness against architecture diagram
- Ensure no feature loss after optimization
- Self-Debugging via .md rules on failure

05 Gradient Correction

- Compare actual result vs Roofline prediction
- Compute error → update CPU/GPU ratio, iterate until convergence

Challenges & Resolutions

- Environment Issues: A.I. might not be able to find the right environment setting and the project won't run.

Solution: manually checking the issue as some information like environmental variables might not be visible to A.I.

- Reproducibility Issues: A.I. might produce results that are far from what are claimed in the paper.

Solution: Compare system specs in the paper to see if there is a huge difference. If not, A.I. might misconfigure somethings. As A.I. might suffer from hallucinations, human intervention is required.

- Self-debugging: feed errors into A.I.
- Reflexion: remember repeating errors, avoid the same error

Reference

- HeteroBench: <https://dl.acm.org/doi/pdf/10.1145/3676151.3719366>